FPGAceTime Preliminary Report

Sanjith Udupa*, Annie Wang*
Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
Cambridge, MA, USA
sanjithu@mit.edu, awang27@mit.edu

***Abstract*—We present FPGAceTime, a bidirectional live audio/video communication system implemented across two RealDigital Urbana FPGAs. Each FPGA captures camera data, downsamples to a 320×180 RGB565 frame, serializes packets onto a differential PMOD link, receives video from the remote board, and composes the final display via HDMI. The communication channel uses a custom 18-bit packetized SPI-like protocol running over TMDS_33 differential pairs driven by OBUFDS/IBUFDS primitives with external 220 Ω pull-up networks. We describe the system architecture, the transmit/receive state machines, serializer/deserializer design, memory organization, and preliminary simulation and oscilloscope evaluation results. This preliminary report reflects all modules implemented to date and outlines remaining integration work.**

I. Introduction

FPGAceTime is a low-bandwidth, real-time audiovisual link implemented on two identical FPGAs. Each board captures live camera frames, performs local downsampling and anti-aliasing for bandwidth reduction, packetizes video data into compact fixed-format packets, transmits these packets over an FPGA-to-FPGA serial link, decodes received packets, and finally composes remote video with a local selfie-view overlay. Figure 5 shows our updated rough block diagram. We explain the different parts throughout this report.

A major design constraint is the limited number of free I/O pins on the Urbana board once the camera is attached, which uses the entire PMOD A header. Only the PMOD B header is available for board-to-board communication, giving eight pins total. To maximize usable bandwidth over this narrow interface, we employ differential signaling using the `OBUFDS` and `IBUFDS` primitives. The corresponding pins are assigned the `TMDS33` I/O standard in the XDC constraints file, and each differential pair is externally supported with a 220 Ω pull-up resistor network to help stabilize the common-mode bias and improve signal integrity at high link speeds.

One of our downstream goals is to include audio transmission, and the report will detail the progress we have made towards this goal. We note that to wire the 4 data pins needed for the ADC which our microphone uses, we used the servo pins on the board, which we remap via the XDC file.

II. Video Capture and Downsampling

The OV5640 camera module produces 720p RGB data at 30 FPS. We currently reconstruct the incoming pixel stream using the Lab 5 low-resolution capture pipeline (`pixel_reconstruct` module), which generates synchronized `h_count`, `v_count`, and valid pixel outputs.

Since the full-resolution stream cannot be transmitted directly (bandwidth limitations described in the next section), we perform downsampling by capturing only every fourth pixel. We write the captured pixels directly into the `tx_frame_buffer` and `selfie_view_buffer` BRAMs.

We intend to swap in the Lab 6 pipeline (which uses DRAM for full-resolution buffering) with minimal structural changes by keeping the interface modular. The current simple subsampling will eventually be replaced with an anti-aliasing stage with a 3x3 box blur filter fed into a 2x downsampler fed into another 3x3 box blur filter fed into another 2x downsampler.

We maintain three BRAM-based framebuffers on each FPGA:

- **tx_frame_buffer**: The 320×180 frame that will be transmitted to the remote FPGA.
- **selfie_view_buffer**: A similarly downsampled frame displayed locally as a selfie inset. Write end is identical to the `tx_frame_buffer`, but the read end is clocked by the `clk_pixel` signal and read for the display composition.
- **rx_frame_buffer**: Stores packets received from the remote board. Read end clocked by the `clk_pixel` signal and read for the display composition.

III. Physical Link and Differential Signaling

The PMOD B header has 8 pins total, which we allocated as four pins for transmission and four pins for reception. Each direction uses differential signaling with positive and negative pairs, yielding two signals per direction: one for data and one for clock. Specifically, the transmit path uses `tx_bit` (differential data) and `tx_dclk` (differential clock), while the receive path uses `rx_bit` (differential data) and `rx_dclk` (differential clock).

We generate the differential clock (`dclk`) by simply counting on the built-in 100 MHz system clock, dividing it down to the desired transmission rate (5 MHz, 12.5 MHz, or 25 MHz). This approach avoids the need for additional clock generation IP blocks that we had originally planned to use, simplifying the design while maintaining sufficient timing accuracy for our

*These authors contributed equally to this work.

link speeds. Our PMOD B pins are also not clock capable, which makes this counter method more appropriate than a generated clock. The clock rate is adjustable to achieve reliable signal transmission.

We set all PMOD B signals designated for differential use to `TMDS33`, enabling the FPGA's internal termination and suitable I/O behavior for high-speed signaling. We implement the differential pairs using `OBUFDS` and `IBUFDS` primitives for transmission and reception, respectively.

We add external pull-up resistor networks to bias each differential pair into a stable region, reducing jitter and promoting clean edges. We tested multiple resistance values through several iterations, optimizing for signal smoothness while ensuring adequate peak-to-peak voltage swing. After evaluation, we settled on 220 Ω pull-up networks, which provide reliable operation at frequencies up to 25 MHz. We discuss this further in Section VIII.

IV. Packet Format and Bandwidth Requirements

We structure each transmitted unit as an 18-bit packet:

$$[\text{2-bit header}]\ [\text{16-bit payload}]$$

We encode the headers as follows:

- **11**: Pixel data
- **01**: Frame-start marker
- **00**: Blanking/padding packet

Given a frame of $320 \times 180 = 57{,}600$ pixels, each transmitted as an 18-bit packet, the absolute minimum link bandwidth required for 10 FPS is:

$$57{,}600 \times 18 \text{ bits} = 1.0368 \text{ Mbits/frame}$$

$$10 \text{ frames/s} \Rightarrow 10.368 \text{ Mbit/s}$$

With additional overhead for frame-start markers and blanking packets, the minimum differential clock rate (`dclk`) must be at least 12.5 MHz to provide sufficient bandwidth for reliable transmission. We tested the link at three different clock rates: 5 MHz, 12.5 MHz, and 25 MHz. Oscilloscope measurements (presented in Section VIII-B) validate signal integrity at all three speeds, demonstrating that the link can operate reliably across this range. Since we can push the clock rate above our requirement of 12.5MHz, we can add redundancy for error correction in the future. With 5 additional bits for each packet, we can implement Hamming single error correction. We will need to determine and evaluate a balance between clock speed and error correction.

V. Transmit and Receive Pipelines

We organized the `tx_video` module as a four-state FSM:

1) **SYNC**: Send repeated sync packets for link alignment. We describe this in more detail later in this section.
2) **FRAME_START**: Emit a marker to indicate a new frame start.
3) **PIXELS**: Stream 57,600 pixel packets directly from `tx_frame_buffer`.
4) **BLANKING**: Emit enough packets to maintain a consistent frame rate before the next frame.

The interface of `tx_video` is designed to be modular: it outputs a lookup index (`lookup_pixel_index`) into the `tx_frame_buffer` BRAM once the packet serializer asserts `ready_for_next_packet`. The serializer maintains a queue of length 2, ensuring that data is always available for transmission without waiting for BRAM read latency. This buffering allows continuous packet streaming even when BRAM access takes multiple clock cycles.

We deliver each packet to the `packet_serializer`, which shifts bits MSB-first onto the differential data line on each rising edge of the transmitted differential clock. The serializer operates in the 100 MHz domain and synchronizes the `tx_clk` signal to prevent metastability. We manage backpressure via the serializer's `ready_for_next_packet` signal.

A. Synchronization Behavior and SYNC State Operation

To achieve reliable frame decoding, both FPGAs must agree on the 18-bit packet boundaries. Because the PMOD link provides only a forwarded differential clock and a single data bit, the receiver has no inherent knowledge of where packet boundaries begin. We resolve this in the SYNC state by transmitting a long, continuous burst of the reserved 18-bit pattern

`FULL_PACKET_SYNC = 18'b101100110101110010`

We chose this pattern to have low autocorrelation, and it should never appear in normal pixel traffic (which uses header bits `11`).

1) **Serializer behavior during SYNC:** During the SYNC state, `tx_video` repeatedly asserts `next_packet_valid` with the sync pattern. The packet serializer then shifts the 18-bit word MSB-first on each rising edge of the differential `tx_clk`. Since the SYNC pattern is repeated thousands of times, the receiver is guaranteed to observe this 18-bit sequence aligned to bit boundaries at some point, even if initially misaligned due to startup conditions or metastability.wh
2) **Deserializer alignment search:** The packet deserializer runs a simple two-state FSM, where the receiver samples the incoming data on the falling edge of the forwarded differential clock to maximize timing margins across the link:
   - *UNALIGNED*: On every `rx_clk` falling edge, the incoming bit is shifted into an 18-bit buffer. After each shift, the buffer contents are compared against `FULL_PACKET_SYNC`. Since SYNC packets arrive continuously, every possible bit phase is tested. When the buffer exactly matches the sync pattern, alignment is declared.
   - *ALIGNED*: Once aligned, the deserializer increments a bit counter with each `rx_clk` falling edge and outputs one valid packet every 18 bits. Packet

boundaries remain stable as long as the forwarded clock is clean and the high-speed link is error-free.

3) **Transition from SYNC to FRAME_START:** While the deserializer is aligning, the receive FSM stays in `ST_SEARCH_SYNC`. When the first valid SYNC packet is emitted, the FSM transitions to `ST_WAIT_FRAME_START`. Only after a correct frame-start header (`01`) is received does the receiver advance to the pixel-capture state.
4) **Why many SYNC packets are transmitted:** The SYNC burst length (`SYNC_COUNT` = 1,000,000) ensures that:
   - The receiver sees many copies of the known sync word,
   - alignment will succeed even if occasional bit errors occur,
   - both sides always begin a frame on a clean packet boundary.

   This design tolerates arbitrary misalignment at power-up, differences in reset timing between boards, and transient link disturbances.
5) **Stability across frames:** After alignment, we do *not* re-synchronize per pixel or per line. Instead, we maintain alignment across all subsequent packets. The SYNC state is only re-entered if the system is reset.

To receive the differential data, we designed a similar path to mirror the transmitter. The `packet_deserializer` captures bits on `rx_clk` edges, performs sync-pattern alignment, and emits completed packets to the `rx_video` FSM.

The `rx_video` module exposes a similar modular interface to the transmitter: it outputs a write index (`fb_write_index`), pixel data (`fb_write_pixel`), and a write enable signal (`fb_write_en`) for writing into the `rx_frame_buffer` BRAM. This interface allows the receive FSM to operate independently of the BRAM implementation details.

The `rx_video` module transitions through the following states:

1) **SEARCH_SYNC**: Detect the repeated sync pattern from the deserializer.
2) **WAIT_FRAME_START**: Wait for the frame-start header (`01`) after detecting sync.
3) **RECEIVE_PIXELS**: Write pixels sequentially into the `rx_frame_buffer` when packets with header `11` are received.
4) **DRAIN_BLANKING**: Wait for blanking packets and transition back to `WAIT_FRAME_START` when a new sync burst is detected.

VI. Display Composition

We use the existing lab pipeline for HDMI rendering. The `display_composer` reads both `rx_frame_buffer` and `selfie_view_buffer` to produce:

- full-screen remote view, or
- picture-in-picture mode with a local selfie overlay.

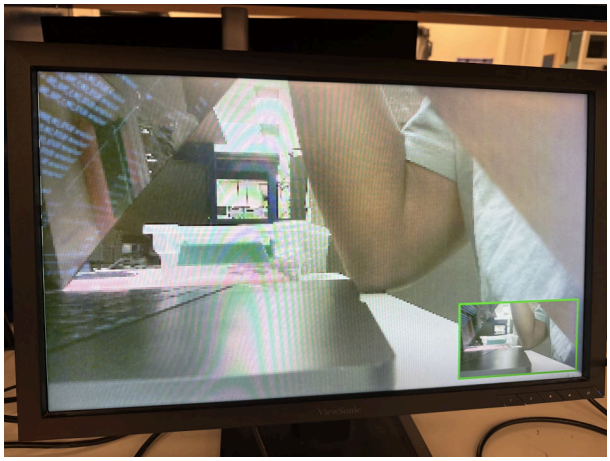


Fig. 1: Display composition showing remote video with selfie view overlay and green border. This test image was generated by having the `rx_frame_buffer` (the larger displayed one) have the same contents as the `selfie_cam_frame_buffer` (smaller one) test the display composition pipeline without the working transmission.

We use a simple coordinate test to determine which frame-buffer to access for each HDMI pixel. As in Figure 1, we position the selfie view in the bottom-right corner with a 20-pixel margin from the screen edges. We render a 5-pixel-wide green border (RGB565 color `0x1ea0`) around the selfie view to visually distinguish it from the remote video feed.

VII. Audio Control

We have implemented audio functionality in separate test modules but have not yet integrated it with the main video transmission and reception state machines. We use a ring buffer interface implemented with a small BRAM to buffer audio samples between capture and transmission.

The BRAM size is calculated based on the target audio specifications: 8 ksps sample rate, 8-bit samples, and a 0.1 s buffer duration (matching the 10 FPS video frame rate). This yields:

$$8000 \text{ samples/s} \times 0.1 \text{ s} = 800 \text{ samples}$$

$$800 \text{ samples} \times 8 \text{ bits} = 6400 \text{ bits} = 800 \times 8 \text{ BRAM}$$

To capture audio, we first implemented the Lab 2 microphone ADC interface, which reads audio samples from the microphone over SPI. On top of this, we added a ring buffer structure with separate read and write pointers to buffer the samples. The write side of the ring buffer stores incoming samples at 8 ksps from the ADC, while the read side supplies samples for transmission. On the receiving FPGA, samples are loaded from the `rx_audio_buffer` ring buffer and output over PWM to the headphone jack. To accommodate the ADC wiring requirements, we connected the four data pins needed for the microphone's SPI interface to the servo pins on the board, which we repurposed for GPIO use via the XDC constraints file. This repurposing was necessary because

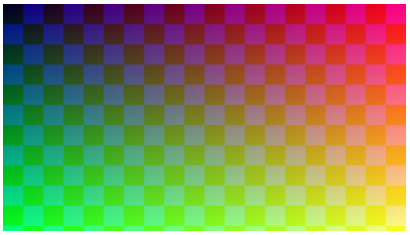


Fig. 2: Image we transmitted in simulation.

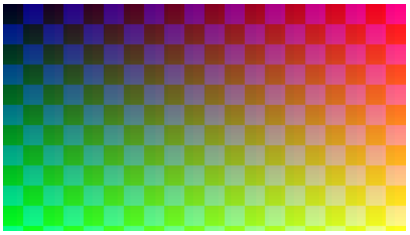


Fig. 3: Simulated reconstruction result. The received image is visually identical to the source image and verified pixel-accurate in code, with minor artifacts from color space conversion

the camera occupies the entire PMOD A header, leaving only PMOD B available for the video link.

For future integration work, we will connect the audio ring buffer's read interface to the `tx_video` state machine for packetization and transmission, and connect the receive path to the `rx_video` state machine for deserialization and playback.

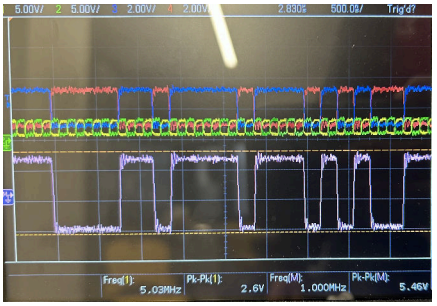
VIII. Evaluation

A. Simulation

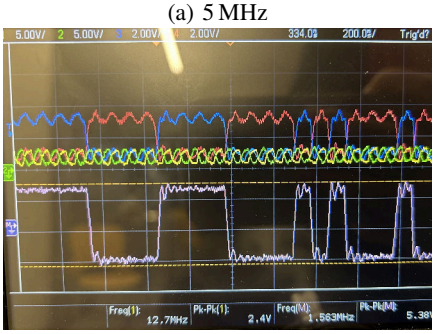
We created a cocotb testbench that instantiates both transmit and receive modules connected by virtual wires (omitting the differential signaling for simplicity in simulation as it just passes through the IPs). The modular interface design allows us to swap the BRAM implementations for Python arrays in the testbench. Specifically, we replace the `tx_frame_buffer` BRAM with a Python array containing the source image, and the `rx_frame_buffer` BRAM with another Python array for the received image.

The testbench leverages the exposed interfaces: `tx_video` outputs `lookup_pixel_index` which we use to read from the Python array, and `rx_video` outputs `fb_write_index`, `fb_write_pixel`, and `fb_write_en` which we use to write into the receive Python array. After transmission completes, we write the received array to an image file for visual comparison.

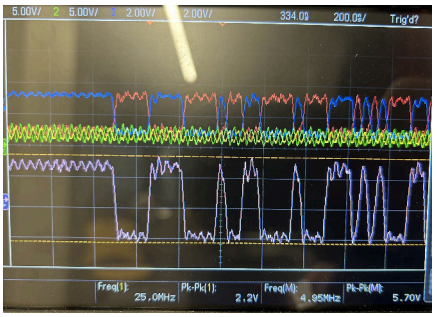
One image we simulated transmitting is in Figure 2, and its reconstruction is in Figure 3. The reconstructed images are visually identical to the source images, with the exception of artifacts visible when zooming in. These artifacts arise from the conversion between the full color space in the source images and the RGB565 format we use in the framebuffers. We verified pixel-level accuracy in code by comparing the transmitted and received pixel arrays, confirming perfect reconstruction of the RGB565 data.



(a) 5 MHz



(b) 12.5 MHz



(c) 25 MHz

Fig. 4: Measured differential waveforms at three transmission rates. The yellow and green traces are the differential clock positive and negative lines, respectively. The blue and red traces are the differential data positive and negative lines, respectively. The purple trace shows the result of subtracting the differential data lines (DATA_P − DATA_N), clearly displaying the digital signal transitions.

B. Oscilloscope Testing

We probed differential clock and data lines at three different transmission rates: 5 MHz, 12.5 MHz, and 25 MHz. We configured the oscilloscope in subtraction mode to cleanly resolve the differential pairs. The purple trace in each capture is the result of subtracting the positive and negative differential data lines, which clearly shows the digital signal transitions. Signal quality remains reliable at all three speeds, though edge jitter becomes more noticeable at 25 MHz. Up to 25 MHz, signal integrity remained sufficient for correct packet decoding.

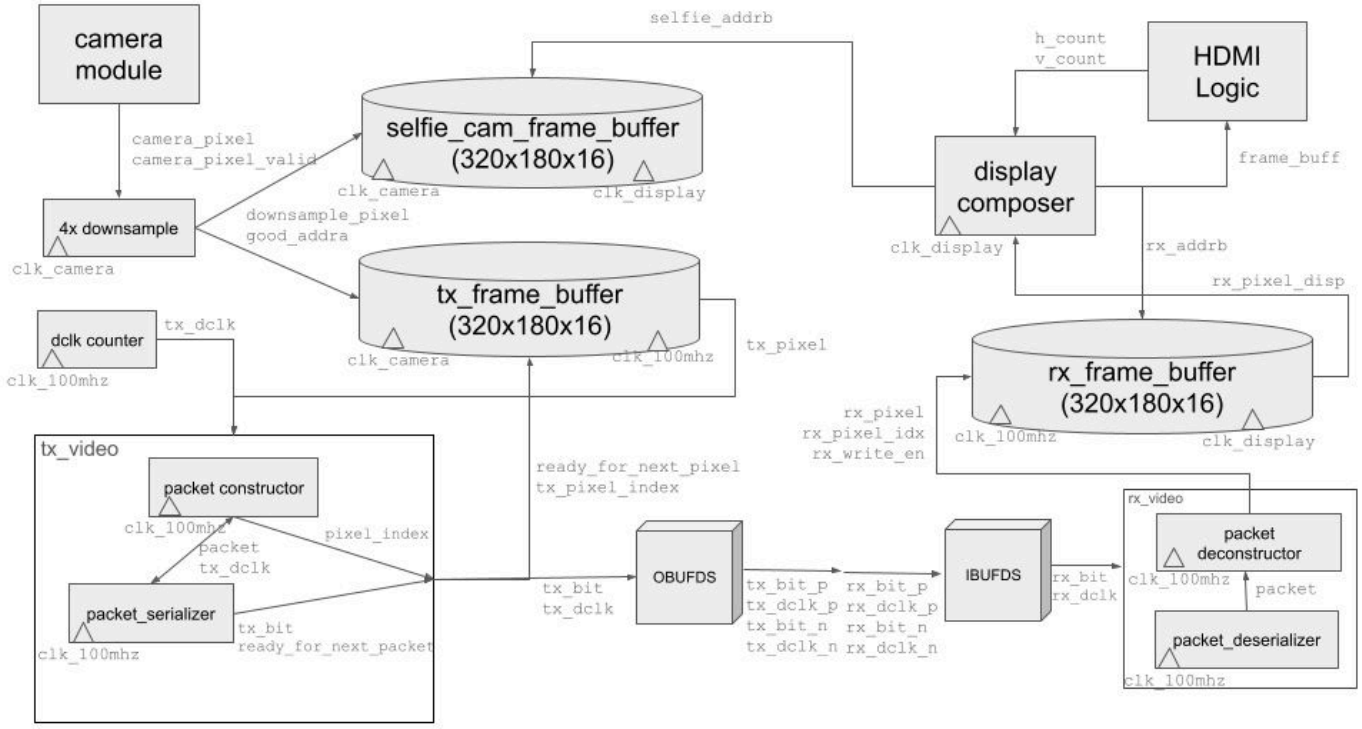


Fig. 5: Rough block diagram explaining the new data flow that is outlined in this report (only video tx/rx as audio is not integrated yet)